

Trabalho Prático I – CCF212

Qual é a fábula? Estudo comparativo entre Árvore PATRICIA e Tabela HASH como estruturas para implementar arquivo invertido num sistema de busca por fábulas

Arthur Emanuel Alecrim Correia [6582]¹, Ryan Italo de Andrade Ananias [6588]¹, Thomaz Augusto Araújo Silva [6577]¹, Alejandro Mateus Escobar Cavalcante [6598]¹

¹ Universidade Federal de Viçosa – Campus Florestal
Disciplina: Algoritmos e Estruturas de Dados II (CCF212) – 2026/1

Este trabalho tem como objetivo a construção de um sistema de busca por fábulas baseado em índice invertido, utilizando duas estruturas de dados distintas: a Árvore PATRICIA e a Tabela Hash com encadeamento. A motivação prática é auxiliar contadores de histórias infantis na busca por fábulas que abordem temas específicos, a partir de palavras-chave fornecidas pelo usuário.

Máquinas de busca, como o Google, operam exatamente sobre esse princípio: ao invés de varrer todos os documentos a cada consulta, os termos são indexados previamente em estruturas que permitem recuperação eficiente. O índice invertido é a estrutura central nesse processo, associando cada palavra a uma lista de documentos nos quais ela aparece, juntamente com sua frequência de ocorrência.

Para calcular a relevância dos documentos retornados, foi utilizado o método TF-IDF (*Term Frequency – Inverse Document Frequency*), que pondera a frequência de um termo em um documento pela raridade desse termo na coleção como um todo.

Os algoritmos de referência utilizados foram os apresentados em sala de aula, baseados em **alguém aí que vcs se basearam** para a Árvore PATRICIA e em [Cormen et al. 2009] para a Tabela Hash.

O código-fonte completo está disponível no repositório do grupo [Correia et al. 2026].

1. Metodologia

O grupo foi organizado de forma que cada integrante ficou responsável por um conjunto de módulos do sistema, apesar de todos os integrantes ficarem cientes da lógica e didática de todas as estruturas utilizadas (para fins de estudo).

- **Thomaz** – Módulo de carregamento do corpus (`loader.c`), lista encadeada de ocorrências (`lista.c`, adaptada de [Ziviani 2010]), estrutura da Tabela Hash (`hash.c`, `hash.h`) e integração geral (`main.c`);
- **Ryan** – Índice invertido da Hash (`indice_hash.c`) e cálculo do TF-IDF da Hash (`tfidf_hash.c`);
- **Arthur** – Árvore PATRICIA (`patricia.c`, `patricia.h`) e índice invertido da Patricia (`indice_patricia.c`);
- **Alejandro** – TF-IDF da Patricia (`tfidf_patricia.c`) e análises comparativas de desempenho.

O compartilhamento de código foi feito via repositório Git, com ramificações por funcionalidade (features), criação de pull requests e integração na branch principal após revisão entre os integrantes.

2. Resultados do Desenvolvimento

2.1. Pré-processamento e Carregamento do Corpus

O módulo `loader.c` é responsável por ler o arquivo `entrada.txt`, que lista os nomes das fábulas a serem indexadas, e carregar o conteúdo de cada uma delas em uma estrutura `Corpus`. Durante o carregamento, cada palavra passa por duas etapas de pré-processamento:

1. **Normalização:** conversão de todas as letras para minúsculas via `tolower()`;
2. **Remoção de pontuação:** eliminação de caracteres não alfabéticos via `isalpha()`.

Após o pré-processamento, palavras classificadas como *stop words* (preposições, artigos e conjunções) são descartadas antes da inserção no corpus. A lista de stop words advinda do PVANET é definida estaticamente em `loader.c` e contém 208 termos.

A estrutura `Corpus` armazena um vetor dinâmico de `Tokens` (palavra + `idDoc`), os nomes dos arquivos e os títulos das fábulas (primeira linha de cada arquivo), além da contagem total de documentos e tokens. O vetor de tokens cresce dinamicamente via `realloc`, dobrando de capacidade quando necessário. Essa estratégia de crescimento exponencial reduz o número de realocações para $O(\log n)$, evitando o custo linear de múltiplos `mallocs` individuais — cada `realloc` pode copiar o bloco inteiro para um novo endereço, então minimizando sua frequência, obtemos um ganho computacional bem interessante.

O corpus processado resultou em **621 tokens** no total (sem stop words, com repetições) e **395 palavras únicas**, distribuídas entre as 20 fábulas.

2.2. Lista Encadeada de Ocorrências

A estrutura `ListaOcorrencias`, definida em `lista.h`, é utilizada por ambos os TADs para armazenar os pares $\langle qtde, idDoc \rangle$ de cada palavra indexada. Cada nó da lista (`Celula`) armazena o identificador do documento e a quantidade de ocorrências da palavra nele.

A função `inserirOcorrencia()` percorre a lista procurando o `idDoc`: se encontrado, incrementa `qtde`; caso contrário, insere uma nova célula no final, mantendo a ordem de inserção (que coincide com a ordem crescente de `idDoc`, pois os documentos são processados sequencialmente).

2.3. Tabela Hash com Encadeamento

2.3.1. Função de Hash

A função de hash adotada é a **polinomial de Horner**:

$$h = \left(\sum_{i=0}^{n-1} c_i \cdot 31^{n-1-i} \right) \mod M \quad (1)$$

onde c_i é o valor ASCII do i -ésimo caractere da palavra e M é o tamanho da tabela. A base 31 foi escolhida por ser prima e pequena o suficiente para não provocar overflow antes da operação realizada sobre `unsigned long`. A escolha dessa função se deu pelo fato de que ela é a mais recomendada para tratar palavras latinas, resultando em um número baixo de colisões.

2.3.2. Dimensionamento da Tabela

O tamanho M da tabela foi definido empiricamente. Após medir o vocabulário real do corpus (395 palavras únicas), aplicou-se o fator de carga alvo $\alpha = 0,75$:

$$M = \left\lceil \frac{n}{\alpha} \right\rceil = \left\lceil \frac{395}{0,75} \right\rceil \approx 527 \quad (2)$$

O primo mais próximo acima de 527 é **541**, valor adotado como `#define M 541` em `hash.h`. A escolha de um número primo reduz colisões porque elimina fatores comuns com a base da função hash, distribuindo os índices de forma mais uniforme.

2.3.3. Tratamento de Colisões

Colisões são tratadas por **encadeamento externo**: cada posição da tabela aponta para uma lista encadeada de `EntradaHash`, onde cada nó armazena uma palavra distinta e sua `ListaOcorrencias`. Com isso, novas entradas são inseridas na **cabeça** da lista.

2.4. Índice Invertido da Hash

O módulo `indice_hash.c` constrói o índice percorrendo todos os tokens do corpus e chamando `inserirHash()` para cada um (`construirIndiceHash`). A impressão em ordem alfabética é feita coletando todos os ponteiros de `EntradaHash` em um vetor auxiliar, ordenando com `qsort` via (`strcmp`), e imprimindo cada entrada seguida de sua lista de ocorrências no formato `<qtde, idDoc>`.

2.5. Cálculo do TF-IDF para a Hash

O cálculo de relevância segue a fórmula TF-IDF definida no enunciado. A implementação está em `tfidf_hash.c` e se divide em duas funções principais: `pesosTermo()`, que busca a palavra na hash, calcula d_j percorrendo a lista de ocorrências e acumula o peso em `soma[idDoc]`; e `tfidf()`, que itera sobre os termos de busca, normaliza por `ni[i]` e ordena os resultados por relevância decrescente via `qsort`.

O vetor `ni[]` é calculado por `calcularNi()`, que percorre a tabela hash já construída em $O(p)$, onde p é o número de palavras únicas, evitando uma solução $O(n^2)$ sobre o corpus.

2.6. Estrutura, Lógica e Otimização da Árvore PATRICIA

A árvore PATRICIA é constituída por nós. Um nó pode ser interno ou pode ser externo, e como ambos possuem armazenamentos similares na memória, foi utilizada a estrutura `union` da linguagem C.

- **Nó Interno:** Armazena o índice que diferiu no processo de comparação entre as palavras, os apontadores para as subárvores do lado esquerdo e do direito, e o caractere que divergiu.
- **Nó Externo (Folha):** Armazena a palavra em si e uma lista encadeada de ocorrências que gerencia e salva os identificadores dos documentos (`idDocs`) e a frequência em que a palavra se repete durante o processo de inserção.

2.7. Lógica PATRICIA e Adaptação

No algoritmo original de Ziviani, a lógica é comumente aplicada para a inserção e roteamento bit a bit ou baseada em caracteres simples. O processo de adaptação neste projeto se deu para o roteamento e inserção de *strings* completas.

Quando a função `Inserer()` é invocada, o algoritmo inicia o percurso na árvore comparando o caractere da palavra nova (na posição do índice armazenado no nó interno atual) com o caractere de roteamento salvo nele. O resultado dessa comparação dita o caminho que a árvore irá seguir: para a esquerda, caso a letra da palavra nova seja menor ou igual à letra do nó; e para a direita, caso seja maior. O algoritmo segue nesse processo de descida até alcançar um nó externo.

Esse nó externo encontrado representa a vizinhança onde a nova palavra deve ser alocada, portanto, a *string* armazenada dentro dele será usada como base de comparação. Vale ressaltar que todas as comparações seguem a ordem alfabética da tabela ASCII. A lógica foi implementada através de uma estrutura de repetição que percorre e compara cada caractere da palavra nova e da palavra armazenada no nó folha. O índice exato em que os caracteres se diferem é retornado e utilizado posteriormente para a criação de uma nova bifurcação.

Listing 1. Algoritmo de comparação de prefixos e identificação de divergência

```
//Algoritmo que compara as palavras
int CompararPalavras (char *palavra, char *palavraArvore, int *qtdComp)
{
    int i = 0;
    while (palavra[i] == palavraArvore[i]){

        (*qtdComp)++;
        i++;

        if (palavra[i] == '\0' && palavraArvore[i] == '\0'){
            return -1;
        }
    }
    return i;
}
```

Em situações de prefixos idênticos (como "casa" e "casamento"), a lógica trata o caractere de terminação de *string* (`\0`) como um caractere válido, ou seja, o algoritmo compara `\0` com a letra 'm' e retorna corretamente o índice da divergência. Quando as palavras comparadas são rigorosamente iguais, a função retorna -1.

Após a comparação, caso o retorno seja -1, o algoritmo aciona a rotina que insere ou atualiza a ocorrência da palavra repetida. Caso contrário, a função `InsererEntre` é

chamada para alocar o novo nó interno e reorganizar os ponteiros da estrutura.

2.8. A Otimização do Menor Caractere e o Roteamento

Um dos grandes diferenciais desta implementação é a decisão arquitetural de sempre armazenar o **menor caractere** no nó interno no exato momento em que duas palavras divergem, utilizando o operador de menor ou igual ($\leq$) para guiar o fluxo à esquerda.

Essa abordagem tira proveito do fato de que o terminador nulo ($\backslash 0$) possui o menor valor numérico possível (zero). Assim, ele atua como um separador de prefixos absoluto. Quando uma palavra que é prefixo de outra termina (por exemplo, "CAR"), o caractere $\backslash 0$ vence qualquer comparação. Isso garante que a *string* base fique sempre isolada e rapidamente acessível na subárvore esquerda. As demais palavras maiores que compartilham o mesmo prefixo (como "CARRO" ou "CARTA") são automaticamente roteadas para a direita, criando subdivisões estruturais em níveis mais profundos apenas entre si.

2.9. O Impacto no Consumo de Memória e Desempenho

O emprego dessa lógica de roteamento influencia de forma direta o comportamento da memória do sistema, mais especificamente o consumo de memória *Stack* (Pilha de Execução).

Do ponto de vista da alocação dinâmica (*Heap*), não há divergência no gasto de memória estrutural: para N palavras distintas, a árvore sempre alocará os mesmos N nós externos e $N - 1$ nós internos, independentemente do critério de roteamento. A otimização, de fato, ocorre na **topologia da árvore**.

Se a lógica inversa fosse adotada (armazenando o maior caractere e percorrendo com $\geq$), as palavras mais curtas seriam continuamente empurradas para a subárvore esquerda. Isso geraria uma topologia degenerada e desbalanceada, semelhante a uma lista encadeada vertical. Consequentemente, as buscas e inserções no final da estrutura exigiriam um número expressivamente maior de chamadas recursivas na memória *Stack*, aumentando o risco de *Stack Overflow* e elevando o tempo de processamento ao analisar corpos de texto volumosos.

Ao utilizar a retenção do menor caractere aliada ao caminho $\leq$, a árvore se distribui de maneira muito mais rasa e horizontalizada. O real ganho de desempenho e memória está na drástica diminuição do uso de recursão, o que reduz simultaneamente os ciclos da CPU e a quantidade total de comparações necessárias para alcançar qualquer folha da estrutura.

2.10. Índice Invertido da Patricia e TF-IDF

Para calcular a relevância de um termo buscado, é preciso primeiramente, achá-lo na árvore. A função `pesquisa` atua como o motor de busca base do sistema. Ela localiza eficientemente uma palavra na estrutura da árvore utilizando o roteamento de caracteres, retornando o nó folha correspondente (e sua respectiva lista de ocorrências) caso haja uma correspondência exata via `strcmp`. Para mensurar o custo computacional, um contador é incrementado a cada nó visitado durante a descida na árvore.

A função `calcular_peso` extrai a relevância de um termo específico dentro da coleção de documentos. Após localizar a palavra, o algoritmo realiza duas iterações sobre a sua lista de ocorrências:

1. **Frequência de Documentos (DF):** Contabiliza em quantos documentos distintos o termo aparece, armazenando o total na variável dj .
2. **Cálculo e Acumulação:** Aplica a métrica de peso para cada documento utilizando a equação:

$$w = qtde \times \left(\frac{\log_2(n)}{dj} \right)$$

Onde $qtde$ é a frequência do termo no documento atual e n é o número total de documentos no *corpus*. O peso w resultante é então acumulado em um vetor global chamado *soma*.

A função `tfidf_patricia` lida com consultas compostas por múltiplos termos, consolidando os pesos de todas as palavras pesquisadas no vetor de acumulação.

Após essa etapa, o algoritmo realiza a normalização: o peso absoluto de cada documento é dividido pela sua quantidade total de palavras (fator ni), gerando a relevância final. Documentos com pontuação válida são armazenados em um vetor de *structs* do tipo *prioridade* e, por fim, ranqueados em ordem decrescente de relevância utilizando o algoritmo `qsort`, que é guiado pela função de comparação `compara_prioridade`.

2.11. Menu e Integração

O `main.c` implementa o menu interativo conforme a Tarefa 5 do enunciado, com as seguintes opções:

1. **Carregar corpus:** lê os arquivos de fábulas via `carregarCorpus()` e exibe a contagem de tokens e documentos;
2. **Construir índices:** inicializa e popula a Hash e a PATRICIA a partir do corpus carregado, e calcula o vetor `ni[]`;
3. **Imprimir índice – HASH:** exibe as palavras em ordem alfabética com suas listas de ocorrências;
4. **Imprimir índice – PATRICIA:**
5. **Buscar fábulas – HASH:** recebe termos de busca, calcula relevância TF-IDF e exibe os resultados ordenados;
6. **Buscar fábulas – PATRICIA:**

2.12. Compilação e Execução

O projeto utiliza um `Makefile` para automatizar a compilação de todos os módulos. A estrutura de arquivos é a seguinte:

```
.
+-- main.c
+-- loader.c / loader.h
+-- lista.c / lista.h
+-- hash/
|   +-- hash.c / hash.h
|   +-- indice_hash.c / indice_hash.h
|   +-- tfidf_hash.c / tfidf_hash.h
+-- patricia/
    +-- patricia.c / patricia.h
    +-- indice_patricia.c / indice_patricia.h
    +-- tfidf_patricia.c / tfidf_patricia.h
```

O Makefile compila cada `.c` em um `.o` separado e os linka em um único executável. A flag `-lm` é passada para disponibilizar a função `log2()` da biblioteca matemática, utilizada no cálculo do TF-IDF. Para compilar e executar:

```
make # compila todos os módulos
make run # executa o sistema
make clean # remove os .o e o executável
```

3. Análises Comparativas

3.1. Comparações nas Operações de Inserção

A Tabela 1 e a Figura 1 apresentam o total de comparações realizadas durante a construção dos índices invertidos em função do número de fábulas indexadas.

Tabela 1. Comparações de inserção – Hash vs. PATRICIA

Fábulas	Tokens	Hash	PATRICIA
1	30	5	242
5	176	60	2383
10	328	135	5022
15	461	219	7520
20	579	320	9858

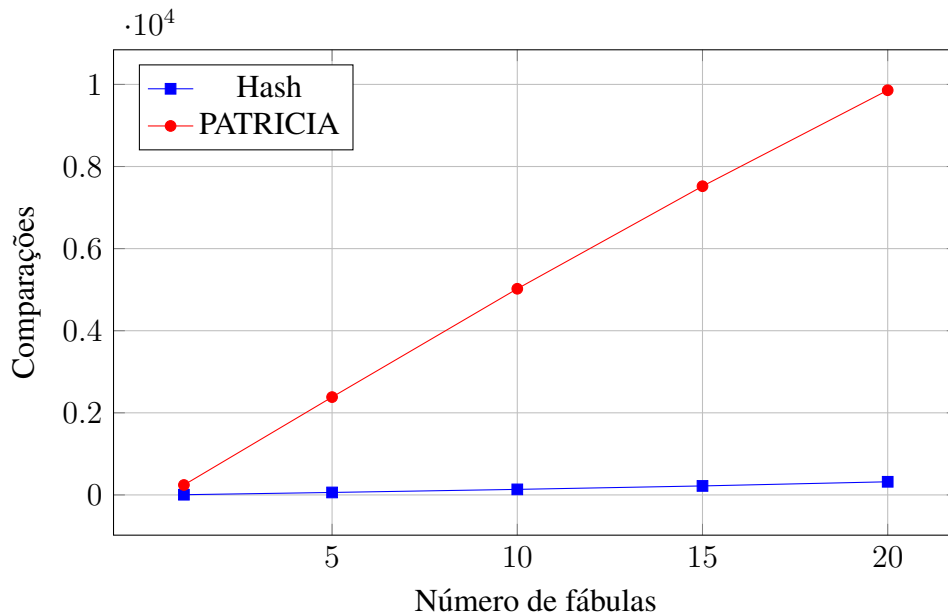


Figura 1. Comparações de inserção em função do número de fábulas.

A Hash realizou aproximadamente 30 vezes menos comparações que a PATRICIA em todos os cenários. O crescimento da Hash é aproximadamente linear em função dos tokens inseridos, com poucas colisões graças ao fator de carga $\alpha = 0,75$ e ao tamanho primo $M = 541$. A PATRICIA também cresce linearmente em tokens, mas com coeficiente muito maior, pois cada inserção percorre $O(\log n)$ nós internos comparando caracteres antes de encontrar a posição correta.

3.2. Comparações nas Operações de Busca

A Figura 2 apresenta as comparações realizadas nas operações de busca para três cenários distintos, utilizando o corpus completo de 20 fábulas.

Tabela 2. Comparações de busca – Hash vs. PATRICIA

Cenário	Termos buscados	Hash	PATRICIA
Melhor caso	colmeia (1 termo)	1	27
Caso médio	lobo, garca, pescoco (3 termos)	3	65
Pior caso	dragao, castelo, principe (3 termos inexistentes)	2	61

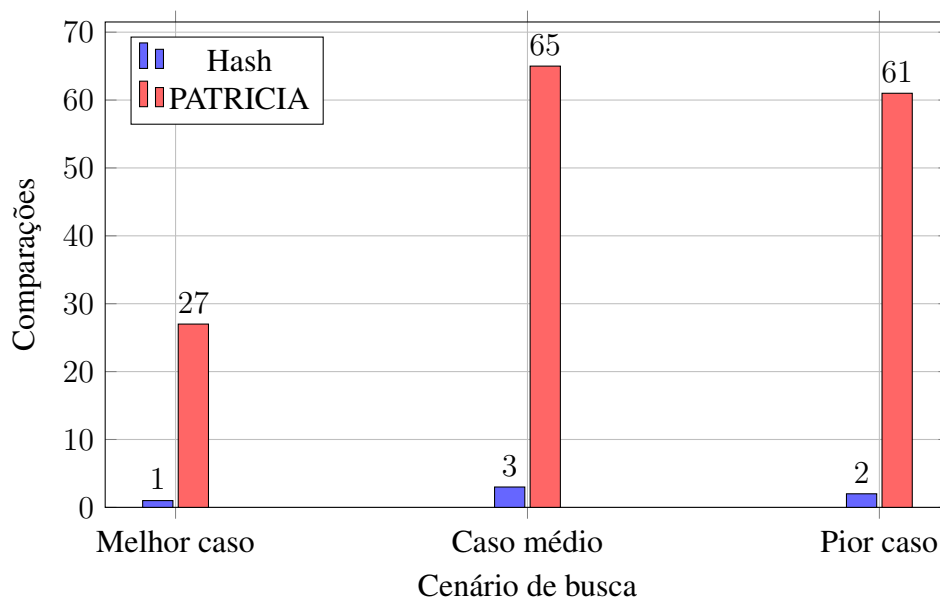


Figura 2. Comparações de busca por cenário – Hash vs. PATRICIA (corpus completo).

Na Hash, o número de comparações por termo é próximo de 1, graças a ausência de colisões nos slots pesquisados. No pior caso, 2 comparações para 3 termos indica que apenas um dos slots tinha uma colisão. Na PATRICIA, o número de comparações por termo é em torno de 20, correspondendo à profundidade média da árvore com 395 palavras únicas ($\log_2 395 \approx 8,6$ níveis). Por incrível que pareça, o pior caso da PATRICIA (61 comparações) foi ligeiramente inferior ao caso médio (65), pois os termos inexistentes terminam a busca assim que chegam a uma folha que não corresponde à palavra, sem percorrer toda a lista de ocorrências.

4. Considerações Finais

Como já visto, na Tabela Hash foi evidenciado um desempenho superior, realizando aproximadamente 30 vezes menos comparações que a PATRICIA na inserção e chegando próximo de 1 comparação por busca na grande parte dos casos.

Contudo, uma das limitações da Hash é a ausência de ordenação natural como estrutura de dados. A impressão em ordem alfabética foi feita a partir da coleta de todos os ponteiros em um vetor auxiliar, ordenado com a função `qsort`, o que ocasiona um custo extra de $O(p \log p)$ que não existiria na PATRICIA, cuja travessia já devolve as palavras em ordem.

A Árvore PATRICIA, por sua vez, demonstrou ser uma estrutura naturalmente ordenada, o que representa uma vantagem em cenários onde a impressão ou varredura lexicográfica do índice é frequente. Sua principal desvantagem ficou evidente nos dados de inserção e busca: o custo por operação é significativamente maior que o da Hash, pois cada inserção percorre $O(\log n)$ nós internos comparando caracteres até encontrar a posição correta. A decisão arquitetural de armazenar o menor caractere no nó interno e utilizar o operador $\leq$ para guiar o roteamento à esquerda contribuiu para uma topologia mais rasa e balanceada, reduzindo o uso de recursão e o risco de *Stack Overflow* em corpos de texto maiores.

Outro ponto favorável à PATRICIA é que ela não exige conhecimento prévio do volume de dados: a árvore cresce dinamicamente, alocando um nó por inserção. A Hash, por outro lado, depende de um M definido a priori — se o vocabulário real for muito maior que o estimado, o fator de carga sobe e o desempenho degrada com o aumento de colisões.

De forma geral, os resultados confirmaram o que a teoria prevê: a Hash é superior em desempenho bruto de inserção e busca, enquanto a PATRICIA oferece vantagem estrutural quando a ordenação das palavras é relevante ou quando o tamanho do vocabulário não é conhecido previamente. Para um sistema de busca como o desenvolvido, onde a recuperação rápida por palavras-chave é o critério central, a Tabela Hash se mostrou a estrutura mais adequada. A implementação conjunta das duas estruturas permitiu comparar na prática os conceitos estudados em sala de aula, evidenciando que a escolha da estrutura de dados depende diretamente das prioridades de cada aplicação.

Referências

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3 edition.
- Correia, A. E. A., Ananias, R. I. d. A., Silva, T. A. A., and Cavalcante, A. M. E. (2026). tp1-aeds2 – implementação de Índice invertido com patricia e hash. <https://github.com/essashr/tp1-aeds2/tree/develop>. Acesso em: jun. 2026.
- Ziviani, N. (2010). *Projeto de Algoritmos com Implementações em Pascal e C*. Cengage Learning, 3 edition.